

Tentu, ini adalah **penjelasan baris demi baris (line-by-line)** yang disusun secara logis. Saya mengelompokkan kode ke dalam blok-blok fungsional agar Anda dan teman Anda tidak pusing membacanya, tetapi tetap mencakup detail setiap baris.

Ini adalah bahan belajar yang sangat baik untuk memahami bagaimana **UI (Tampilan)** berinteraksi dengan **Logic (Otak Program)**.

1. Header & Import Library

Bagian ini memberitahu program di mana harus mencari alat-alat yang dibutuhkan.

C#

```
using MiniPhotoshop.Forms;      // Mengambil form lain (jika ada)
using MiniPhotoshop.Helpers;    // Mengambil alat bantu umum
using MiniPhotoshop.Logic;      // Mengambil logika utama
using MiniPhotoshop.Logic.Helpers; // Mengambil helper logika (seperti koordinat)
using MiniPhotoshop.Logic.Histogram; // Mengambil logika histogram
using MiniPhotoshop.Logic.ImageProcessing; // Mengambil logika pemrosesan citra
using MiniPhotoshop.Logic.IO;   // Mengambil logika Input/Output (Save/Load)
using System.Collections.Generic; // Mengizinkan penggunaan List dan Dictionary
using System.Drawing;           // Inti grafis C# (Bitmap, Color, Graphics)
using System.Drawing.Drawing2D; // Fitur grafis lanjut
using System.Linq;              // Fitur query data (jarang dipakai di grafis murni, tapi ada di template)
using System.Windows.Forms;     // Dasar dari aplikasi Windows Form
using MiniPhotoshop.Logic.Geometry; // Mengambil logika rotasi, scaling, translasi
using MiniPhotoshop.Logic.EdgeDetection; // Mengambil logika deteksi tepi (Canny, Sobel, dll)
```

2. Deklarasi Kelas & Variabel Global

Di sini kita menyiapkan "memori" yang akan diingat selama Form terbuka.

C#

```
namespace MiniPhotoshop
```

```

{
    public partial class Form1 : Form
    {
        // Service utama untuk edit gambar. 'readonly' artinya hanya diisi sekali saat start.
        private readonly ImageEditorService _editorService;

        // Manager untuk mengatur tombol mana yang boleh diklik (Enabled/Disabled).
        private readonly UIManager _toolManager;

        // Manager khusus untuk fitur drag & drop gambar.
        private readonly DragDropManager _dragDropManager;

        // Manager untuk operasi matematika (+, -, *, /).
        private readonly OperationManager _operationManager;

        // Kamus untuk memetakan RadioButton ke PictureBox thumbnail (agar kodenya rapi).
        private Dictionary<RadioButton, PictureBox> _thumbnailMap;

        // Penanda (Flag): Apakah user sedang dalam mode memilih warna?
        private bool isColorSelectionMode = false;

        // Penanda: Apakah user sedang dalam mode isolasi objek (Pseudo Color)?
        private bool _isIsolationMode = false;

        // Tempat menyimpan cadangan gambar asli (untuk fitur Undo/Reset saat drag).
        private Bitmap _tempOriginalImage;

        // Nilai toleransi warna untuk seleksi (default 60).
        private int _currentTolerance = 60;

        // Variabel untuk menyimpan potongan gambar yang sedang di-drag.
        private Bitmap _imageToDrag = null;
    }
}

```

3. Constructor (Fungsi Form1())

Kode ini jalan otomatis **satu kali** saat aplikasi pertama kali dibuka.

C#

```

public Form1()
{
    InitializeComponent(); // Menyiapkan semua tombol dan layout visual.

    pictureBoxHistogram.AllowDrop = true; // Mengizinkan kotak kanan menerima "drop" barang.

    // Mendaftarkan Event: Apa yang terjadi saat barang masuk & dilepas di kotak kanan.
    pictureBoxHistogram.DragEnter += PictureBoxHistogram_DragEnter;
    pictureBoxHistogram.DragDrop += PictureBoxHistogram_DragDrop;

    // Mendaftarkan Event: Apa yang terjadi saat mouse bergerak di kotak kiri.
    pictureBox1.MouseMove += PictureBox1_MouseMove;

    // Mengisi kamus: Menghubungkan tombol bulat (RadioButton) dengan kotak gambar kecilnya.
    _thumbnailMap = new Dictionary<RadioButton, PictureBox>
    {
        { radioButtonThum1, thumbPictureBox1 },
        { radioButtonThum2, thumbPictureBox2 },
        { radioButtonThum3, thumbPictureBox3 },
        { radioButtonThum4, thumbPictureBox4 }
    };

    // Menghidupkan otak pemrosesan gambar.
    _editorService = new ImageEditorService();

    // Membuat daftar tombol yang akan dikunci saat belum ada gambar.
    var toolsToManage = new List<Control>
    {
        button2, button3, button4, button5, button6, button7,
        button8, button10, trackBarBlackWhite, trackBarBrightness,
    };

    // Menghidupkan UIManager dan memberinya daftar alat tadi.
    _toolManager = new UIManager(
        toolsToManage,
        trackBarBrightness,
        lblBrightnessValue,
        trackBarBlackWhite
    );

    // Mematikan tombol-tombol tersebut (karena belum ada gambar saat start).
    _toolManager.DisableTools();

```

```

fileToolStripMenuItem.Enabled = false; // Menu file juga dimatikan dulu.

// Membuat daftar kotak thumbnail.
var thumbnails = new List<PictureBox>
{
    thumbPictureBox1, thumbPictureBox2, thumbPictureBox3, thumbPictureBox4
};

// Menghidupkan DragDropManager untuk mengatur perpindahan gambar dari thumbnail ke
utama.
_dragDropManager = new DragDropManager(
    pictureBox1,
    thumbnails,
    pictureBoxHistogram,
    _editorService,
    _toolManager,
    fileToolStripMenuItem
);

// Mengaktifkan event klik/drag pada thumbnail.
_dragDropManager.RegisterDragDropEvents();

// Menghidupkan OperationManager untuk matematika citra.
_operationManager = new OperationManager(_editorService, _thumbnailMap,
txtConstantValue);

// -- PENGATURAN SHORTCUT EVENT (Lambda Expression) --
// Tombol 5 diklik -> Ambil Channel Merah (0) -> Tampilkan.
button5.Click += (s, e) => { if (_editorService.IsImageLoaded) pictureBox1.Image =
_editorService.GetChannel(0); };
// Tombol 6 -> Channel Hijau (1).
button6.Click += (s, e) => { if (_editorService.IsImageLoaded) pictureBox1.Image =
_editorService.GetChannel(1); };
// Tombol 7 -> Channel Biru (2).
button7.Click += (s, e) => { if (_editorService.IsImageLoaded) pictureBox1.Image =
_editorService.GetChannel(2); };
// Tombol 4 -> Ubah ke Grayscale.
button4.Click += (s, e) => { if (_editorService.IsImageLoaded) pictureBox1.Image =
_editorService.ApplyGrayscale(); };

// Menu Histogram diklik -> Panggil fungsi tampilkan histogram sesuai channel.
histogramRToolStripMenuItem.Click += (s, e) => TampilkanHistogramChannel(0);
histogramGToolStripMenuItem.Click += (s, e) => TampilkanHistogramChannel(1);

```

```

        histogramBToolStripMenuItem.Click += (s, e) => TampilkanHistogramChannel(2);
        histogramGrToolStripMenuItem.Click += (s, e) => TampilkanHistogramChannel(3);

        // Menghubungkan tombol fisik lainnya ke fungsi masing-masing di bawah.
        this.button8.Click += new System.EventHandler(this.button8_Click);
        this.pictureBox1.MouseClick += new
System.Windows.Forms.MouseEventHandler(this.pictureBox1_MouseClick);
        this.button10.Click += new System.EventHandler(this.btnNegation_Click);
        this.trackBarBlackWhite.Scroll += new
System.EventHandler(this.trackBarBlackWhite_Scroll);
        this.trackBarBrightness.Scroll += new
System.EventHandler(this.trackBarBrightness_Scroll);
    }

```

4. Helper & Mouse Click

Fungsi bantuan untuk merapikan UI dan seleksi warna.

C#

```

        // Fungsi untuk menormalkan tampilan (mematikan mode aneh-aneh).
        private void ResetUIState()
        {
            isColorSelectionMode = false; // Matikan mode seleksi.
            pictureBox1.Cursor = Cursors.Default; // Ubah kursor jadi panah biasa.
            _toolManager.ResetControls(); // Minta Manager mereset tombol.
        }

        // Event saat user klik gambar (untuk seleksi warna).
        private void pictureBox1_MouseClick(object sender, MouseEventArgs e)
        {
            // Cek: Kalau bukan mode seleksi ATAU gambar belum ada, stop.
            if (!isColorSelectionMode || !_editorService.IsImageLoaded) return;

            // Ubah koordinat Mouse (di layar) menjadi koordinat Pixel (di gambar asli).
            Point imagePoint = CoordinateHelper.TranslateMouseClickToImagePoint(pictureBox1,
e.Location);

```

```
// Lakukan seleksi warna di titik tersebut lewat Service.
pictureBox1.Image = _editorService.ApplyColorSelection(imagePoint);

// Setelah selesai, matikan mode seleksi & kembalikan kursor.
isColorSelectionMode = false;
pictureBox1.Cursor = Cursors.Default;
}
```

5. Bagian Histogram & Equalization

Logika untuk menampilkan grafik dan memperbaiki kontras.

C#

```
#region Histogram
private void TampilkanHistogramChannel(int channel)
{
    if (!_editorService.IsImageLoaded) { /* Tampilkan pesan error jika gambar kosong */ return; }

    // Minta Service membuatkan gambar grafik batang seukuran PictureBox kanan.
    Bitmap histBmp = _editorService.GetHistogram(channel, pictureBoxHistogram.Width,
    pictureBoxHistogram.Height);

    // Tempel gambar grafik itu ke PictureBox kanan.
    pictureBoxHistogram.Image = histBmp;
}

// Menu: Adaptive Equalization
private void adaptiveEqualizationToolStripMenuItem_Click(object sender, EventArgs e)
{
    if (!_editorService.IsImageLoaded) { /* Error handling */ return; }

    HistogramManager manager = new HistogramManager(); // Panggil ahli histogram.
    this.Cursor = Cursors.WaitCursor; // Ubah kursor jadi jam pasir (Loading).

    try
    {
        // Proses gambar dengan metode Adaptive Equalization.
    }
}
```

```

        Bitmap result = manager.ProcessAdaptiveEqualization((Bitmap)pictureBox1.Image);
        pictureBox1.Image = result; // Tampilkan hasil.
    }
    catch (Exception ex) { MessageBox.Show("Error: " + ex.Message); }
    finally
    {
        this.Cursor = Cursors.Default; // PENTING: Kembalikan kursor walau error.
    }
}

// (LinearStretch dan GlobalEqualization memiliki pola kode yang SAMA PERSIS dengan di atas)
// Mereka hanya memanggil metode manager yang berbeda (ProcessLinearStretch /
ProcessGlobalEqualization).
#endregion

```

6. Bagian Manipulasi (Geometri)

Kode untuk translasi, distorsi, zoom, dan rotasi.

C#

```

#region Manipulation Menu Events
// Menu: Translasi (Geser)
private void translasiToolStripMenuItem_Click(object sender, EventArgs e)
{
    if (!_editorService.IsImageLoaded) { /* Error handling */ return; }

    Translation transLogic = new Translation(); // Panggil ahli translasi.
    // RequestTranslation akan membuka Dialog Input, lalu memproses gambar.
    Bitmap result = transLogic.RequestTranslation((Bitmap)pictureBox1.Image);

    if (result != null) pictureBox1.Image = result; // Update jika ada hasil.
}

// Menu: Distorsi (Miring/Penyok) -> Logikanya sama dengan Translasi.
private void distorsiToolStripMenuItem_Click(object sender, EventArgs e) { ... }

// Menu: Zoom In

```

```

private void zoomInToolStripMenuItem_Click(object sender, EventArgs e)
{
    if (!_editorService.IsImageLoaded) { ... }
    Scaling scaleLogic = new Scaling();
    // Panggil proses scaling dengan faktor 2.0 (200%).
    Bitmap result = scaleLogic.ProcessScaling((Bitmap)pictureBox1.Image, 2.0f);
    pictureBox1.Image = result;
}

// Menu: Zoom Out -> Sama, tapi faktornya 0.5f (50%).
#endregion

```

7. Tombol Load, Save, Restore

Manajemen file dan reset.

C#

```

// Tombol 1: Load Gambar
private void button1_Click(object sender, EventArgs e)
{
    OpenFileDialog openFileDialog = new OpenFileDialog();
    openFileDialog.Filter = "Image Files|*.bmp;*.jpg;..."; // Filter jenis file.

    if (openFileDialog.ShowDialog() == DialogResult.OK) // Jika user pilih file & klik OK.
    {
        var loadedImage = new Bitmap(openFileDialog.FileName);
        // Serahkan ke Manager untuk dimuat ke thumbnail & ditampilkan.
        _dragDropManager.LoadImageToThumbnail(loadedImage);
    }
}

// Tombol 2: Save Data Pixel ke TXT
private void button2_Click(object sender, EventArgs e)
{
    // Cek ada gambar tidak.
    if (pictureBox1.Image == null) { ... return; }
}

```



```

        Bitmap bmpToSave = new Bitmap(pictureBox1.Image);

        using (SaveFileDialog sfd = new SaveFileDialog())
        {
            sfd.Filter = "File teks (*.txt) | *.txt";
            if (sfd.ShowDialog() == DialogResult.OK)
            {
                try {
                    // Panggil FileExporter untuk menulis angka-angka pixel ke file.
                    FileExporter.ExportPixelDataToTxt(bmpToSave, sfd.FileName);
                    MessageBox.Show("Sukses...");
                }
                catch (Exception ex) { ... }
            }
        }

        // Tombol Restore (Reset Total)
        private void buttonRestore_Click(Object sender, EventArgs e)
        {
            if (!_editorService.IsImageLoaded) return;

            // 1. Ambil gambar asli yang masih bersih dari Service.
            pictureBox1.Image = _editorService.GetRestoredImage();
            ResetUIState(); // Reset kursor & mode.

            // 2. Hapus backup sementara (tempOriginal) dari memori.
            if (_tempOriginalImage != null) { _tempOriginalImage.Dispose(); _tempOriginalImage = null; }

            // 3. Hapus gambar di kanan (histogram/hasil drop).
            if (pictureBoxHistogram.Image != null) { pictureBoxHistogram.Image.Dispose(); pictureBoxHistogram.Image = null; }

            // 4. Hapus sisa potongan gambar di memori drag.
            if (_imageToDrag != null) { _imageToDrag.Dispose(); _imageToDrag = null; }
        }

```

8. Fitur Drag & Drop (Pseudo Color / Isolasi Objek) - Bagian Tersulit

Bagian ini mengatur logika "Gunting dan Tempel" objek antar PictureBox.

C#

```
#region Pseudo Color
```

```
// Tombol Toggle Mode Isolasi
```

```
private void PseudoButton_Click(object sender, EventArgs e)
```

```
{
```

```
    if (pictureBox1.Image == null) { ... return; }
```

```
    _isIsolationMode = !_isIsolationMode; // Balikkan status (Hidup <-> Mati).
```

```
    if (_isIsolationMode) // Jika jadi Aktif
```

```
    {
```

```
        pictureBox1.Cursor = Cursors.Cross; // Kursor jadi tanda tambah.
```

```
        PseudoButton.Text = "Matikan Mode";
```

```
        PseudoButton.BackColor = Color.LightGreen;
```

```
    }
```

```
    else // Jika jadi Mati
```

```
    {
```

```
        pictureBox1.Cursor = Cursors.Default; // Kursor normal.
```

```
        PseudoButton.Text = "Mode Isolasi";
```

```
        PseudoButton.BackColor = SystemColors.Control;
```

```
        // Kembalikan gambar asli jika masih ada backup tersimpan.
```

```
        if (_tempOriginalImage != null) {
```

```
            pictureBox1.Image = _tempOriginalImage;
```

```
            _tempOriginalImage = null;
```

```
        }
```

```
    }
```

```
}
```

```
// 1. Mouse ditekan di gambar (Mulai Seleksi)
```

```
private void pictureBox1_MouseDown(object sender, MouseEventArgs e)
```

```
{
```

```
    // Validasi syarat (Mode aktif? Gambar ada? Klik Kiri?).
```

```
    if (!_isIsolationMode) return;
```

```
    if (pictureBox1.Image == null) return;
```

```
    if (e.Button != MouseButtons.Left) return;
```

```
    // Backup gambar asli ke variabel _tempOriginalImage (hanya jika belum ada).
```

```

        if (_tempOriginalImage == null) {
            _tempOriginalImage = (Bitmap)pictureBox1.Image.Clone();
        }

        // Ambil koordinat pixel yang diklik.
        Point imgPoint =
MiniPhotoshop.Logic.Helpers.CoordinateHelper.TranslateMouseClickedToImagePoint(pictureBox
1, e.Location);

        // Validasi koordinat (jangan sampai klik di luar batas gambar).
        if (imgPoint.X < 0 || ... ) return;

        // Ambil warna pixel di titik tersebut.
        Color targetColor = _tempOriginalImage.GetPixel(imgPoint.X, imgPoint.Y);
        var helper = new MiniPhotoshop.Logic.Helpers.PseudoColorHelper();

        // Tampilkan PREVIEW (objek terisolasi background abu-abu).
        Bitmap previewResult = helper.IsolateColor(_tempOriginalImage, targetColor,
_currentTolerance);
        pictureBox1.Image = previewResult;

        // SIAPKAN BARANG DRAG: Ekstrak objek dengan background transparan.
        if (_imageToDrag != null) _imageToDrag.Dispose(); // Buang sampah lama.
        _imageToDrag = helper.ExtractObject(_tempOriginalImage, targetColor,
_currentTolerance);
    }

    // 2. Mouse dilepas (Batal Drag / Selesai Klik)
    private void pictureBox1_MouseUp(object sender, MouseEventArgs e)
    {
        // Jika user cuma klik (tidak drag), kembalikan tampilan ke normal (Original).
        if (_isIsolationMode && _tempOriginalImage != null)
        {
            pictureBox1.Image = _tempOriginalImage;
            _tempOriginalImage = null; // Reset backup agar klik berikutnya ambil baru.
        }
    }

    // Slider Toleransi digeser
    private void trackBarTolerance_Scroll(object sender, EventArgs e)
    {
        _currentTolerance = trackBarTolerance.Value; // Update nilai toleransi.
        lblTolerance.Text = "Toleransi: " + _currentTolerance.ToString(); // Update label.
    }

```

```

    }

    // 3. Mouse digeser (Proses Dragging berjalan)
    private void PictureBox1_MouseMove(object sender, MouseEventArgs e)
    {
        // Syarat Drag: Klik Kiri Tahan + Mode Isolasi + Barang (_imageToDrag) sudah siap.
        if (e.Button == MouseButtons.Left && _isIsolationMode && _imageToDrag != null)
        {
            // MULAI Windows DragDrop. Ini akan mem-pause kode ini sampai drop selesai.
            // Efeknya 'Move' (Pindah).
            DragDropEffects effect = pictureBox1.DoDragDrop(_imageToDrag,
            DragDropEffects.Move);

            // --- LOGIKA SETELAH DROP SELESAI ---
            // Jika penerima bilang "Oke, barang sudah diterima (Move)"
            if (effect == DragDropEffects.Move)
            {
                var helper = new MiniPhotoshop.Logic.Helpers.PseudoColorHelper();
                // Hapus objek dari gambar asli (jadi bolong/putih).
                helper.EraseObject(_tempOriginalImage, _imageToDrag);

                // Update tampilan PictureBox1 dengan versi bolong.
                pictureBox1.Image = (Bitmap)_tempOriginalImage.Clone();
            }
        }
    }

    // 4. Barang masuk ke wilayah PictureBox Histogram (Kanan)
    private void PictureBoxHistogram_DragEnter(object sender, DragEventArgs e)
    {
        // Cek apakah yang dibawa itu Gambar (Bitmap).
        if (e.Data.GetDataPresent(DataFormats.Bitmap))
            e.Effect = DragDropEffects.Move; // Izinkan.
        else
            e.Effect = DragDropEffects.None; // Tolak.
    }

    // 5. Barang dilepas di PictureBox Histogram (Kanan)
    private void PictureBoxHistogram_DragDrop(object sender, DragEventArgs e)
    {
        // Ambil paket gambar yang dikirim.
        Bitmap receivedImage = (Bitmap)e.Data.GetData(DataFormats.Bitmap);
        if (receivedImage == null) return;
    }

```

```

        Bitmap finalResult;

        // Logika Penggabungan (Compositing)
        if (pictureBoxHistogram.Image == null)
        {
            // KASUS A: Kanvas Kosong -> Buat baru, cat putih, tempel objek.
            finalResult = new Bitmap(receivedImage.Width, receivedImage.Height,
System.Drawing.Imaging.PixelFormat.Format24bppRgb);
            using (Graphics g = Graphics.FromImage(finalResult))
            {
                g.Clear(Color.White);
                g.DrawImage(receivedImage, 0, 0);
            }
        }
        else
        {
            // KASUS B: Sudah ada isi -> Ambil gambar lama, tempel objek baru di atasnya (Overlay).
            finalResult = new Bitmap(pictureBoxHistogram.Image);
            using (Graphics g = Graphics.FromImage(finalResult))
            {
                // Tidak di-Clear agar gambar lama tetap ada.
                g.DrawImage(receivedImage, 0, 0);
            }
        }

        // Tampilkan hasil gabungan.
        pictureBoxHistogram.Image = finalResult;
        pictureBoxHistogram.SizeMode = PictureBoxSizeMode.Zoom;

        // Beri sinyal 'Move' ke pengirim (PictureBox1) agar dia menghapus objek asalnya.
        e.Effect = DragDropEffects.Move;
    }
    #endregion

```

Tips Penjelasan Ujian

Saat menjelaskan per baris, Anda tidak perlu membaca sintaksnya (seperti "kurung kurawal buka, if, kurung buka..."). Fokuslah pada **tujuannya**:

- Contoh untuk using (Graphics g = ...): "Baris ini membuat alat lukis (Graphics) untuk memanipulasi gambar finalResult. Kita pakai using agar alat ini otomatis dibuang dari

memori setelah selesai melukis."

- Contoh untuk DoDragDrop: "Baris ini memerintahkan Windows untuk memulai proses seret-gambar. Kode akan menunggu di sini sampai user melepas klik mouse-nya di tempat lain."